UNIVERSITY OF HULL

Principles of distributed programming

UNIVERSITY OF HULL

Distributed Applications

- A distributed application is:
  - an application that spreads its execution over more than one computer
- where
  - individual distributed components interact via the network
  - individual distributed components have the potential to execute in parallel

UNIVERSITY OF HULL

Why Distributed Applications?

- Two key criteria for distributed applications
  - Performance
    - can service thousands of simultaneous users
  - Scalability
    - can be scaled-up to cope with more users or requests
- Other reasons for using distributed architectures
  - resource sharing, reliability and fault tolerance
  - integrating existing legacy systems
  - providing synchronisation and communication between multiple clients (e.g. multiplayer gaming system)

UNIVERSITY OF HULL

What is a Distributed Application?

- What are their distinguishing features of distributed applications?
  - Usage patterns and volume of work
  - Networking and communication
  - Hardware and software platforms
  - Performance
  - Concurrency and consistency
  - Transparency
  - Time
  - Reliability and fault tolerance
  - Security
  - Scalability

UNIVERSITY OF HULL

Example MMORPG

- A MMORPG is an extreme example of a distributed system
  - 10M+ subscribers (e.g. World of Warcraft, Final Fantasy, Elder Scrolls, …)
  - Characteristics
    - Reliable
    - Scalable
    - Secure
    - High performance

UNIVERSITY OF HULL

1. Usage patterns

- Number of users, volume and distribution of requests varies with
  - Specific application or service
  - Time
- Workload and patterns of usage affect design, performance and scalability of a distributed application
- e.g. Web servers
  - Some service millions of request per day
  - Others have only a few requests per day

UNIVERSITY OF HULL

2. Networking and Communication

- Main motivation for distributed systems?
  - to share resources and information
- This requires a network with components & resources located at network nodes
- Networking issues
  - bandwidth variations
    - some clients/parts of system have high-bandwidth connections, others are low-bandwidth
    - requirements vary with type of application
      - e.g. streaming media => high bandwidth
  - connectivity variations
    - some clients have permanent connections, others (e.g. mobile clients) connect intermittently

UNIVERSITY OF HULL

3. Hardware/Software Platforms

- Distributed applications often support a variety of different hardware and software platforms with widely varying characteristics
- For example, Web application must operate correctly with different
  - client implementations
    - e.g. Edge, Firefox, Chrome, etc
  - operating systems
    - e.g. Windows, Android, Linux, etc
  - hardware platforms
    - e.g. x86, x64, ARM-based mobile devices
  - programming languages
    - e.g. Java server, C++ client
  - device characteristics
    - e.g. 4k screen on one device, 1080p on another
- Middleware used to provide common programming abstraction and mask heterogeneity

UNIVERSITY OF HULL

4. Performance

- Two key aspects of distributed application development
- Well-designed distributed app should
  - Be capable of dealing with thousands of simultaneous clients
  - Easily scaled to cope with increased users or resources

UNIVERSITY OF HULL

5. Concurrency and Consistency

- Concurrency is the norm in networked systems
  - servers interact with multiple clients
  - multiple clients may access a resource simultaneously
- Concurrency needs to be controlled to ensure
  - resources are shared correctly (and fairly?)
  - all clients have a consistent view of the state of system objects and resources
  - desired throughput and performance are achieved
- Concurrency control mechanisms and transactions can help
  - monitors, semaphores, locks, mutex's, etc

UNIVERSITY OF HULL

6. Transparency

- Concealing separation of distributed components from user/programmer so system is perceived as a whole
  - Access transparency
    - local and remote resources used the same way
  - Location transparency
    - access to resources without knowledge of location => naming
  - Concurrency transparency
    - multiple processes operate concurrently without interference
  - Replication transparency
    - multiple resources instances used to increase performance/reliability without user knowledge
  - Failure transparency
    - concealment of faults and continued operation
  - Mobility transparency
    - movement of resource/clients within the system
  - Performance transparency
    - reconfiguration of system as loads vary
  - Scaling transparency
    - system/applications expandable in scale without changes to system structure or algorithms

UNIVERSITY OF HULL

7. Time

- Close coordination between distributed components requires shared notion of time
- Clock synchronisation needed to
  - order activities and events using timestamps
  - maintain data consistency
  - eliminate duplicate requests/updates
  - check authenticity of requests
- Limits to accuracy with which computer clocks can be synchronized
- Difficult to determine state of distributed computations due to lack of global clock

UNIVERSITY OF HULL

8. Failure & Fault Tolerance

- Distributed systems experience partial failures
  - Some components fail, others do not
- Distributed apps must deal with failures through
  - Failure detection
    - e.g. checksums, liveness tests, etc
    - not always possible to detect a failures; how to cope when failure is suspected?
  - Failure masking
    - hide effects and lessen their severity
  - Fault tolerance
    - Use of redundancy to allow continued operation in presence of failed components
  - Failure recovery
    - Use of persistent data to restore system to known good state following failure

UNIVERSITY OF HULL

9. Security and Privacy

- Security of resources and privacy of user data is critical, especially when public networks are involved
- Three key aspects
  - Confidentiality
    - protection against unauthorized disclosure
  - Integrity
    - protection against corruption/alteration
  - Availability
    - protection against interference to means of accessing a resource
- Security is not just concerned with concealing data content, but also with authenticating users
  - Preventing Denial of Service attacks

UNIVERSITY OF HULL

10. Scalability

- A system is scalable if it remains effective when number of resources/users increases significantly
- Scalability is a key criteria of many distributed applications
- Scalable system design requires
  - Controlling cost of physical resources
    - possible to extend system at reasonable cost
  - Controlling performance loss
    - max. performance loss should be less than O(log n)
    - hierarchical structures better than linear structures
  - Preventing resource exhaustion
  - Avoiding performance bottlenecks
    - decentralization, replication and caching

UNIVERSITY OF HULL

Designing Distributed Applications

- Designing good distributed applications requires
  - Good models for predicting performance
  - Adherence to design principles
    - Principle of Locality
      - related parts of app located in close proximity
      - data kept close to where it is used
    - Principle of Sharing
      - resources shared to minimize load
    - Principle of Parallelism
      - deploying load across multiple servers enables scaling
  - Design for reliability

UNIVERSITY OF HULL

Distributed Architectures

UNIVERSITY OF HULL

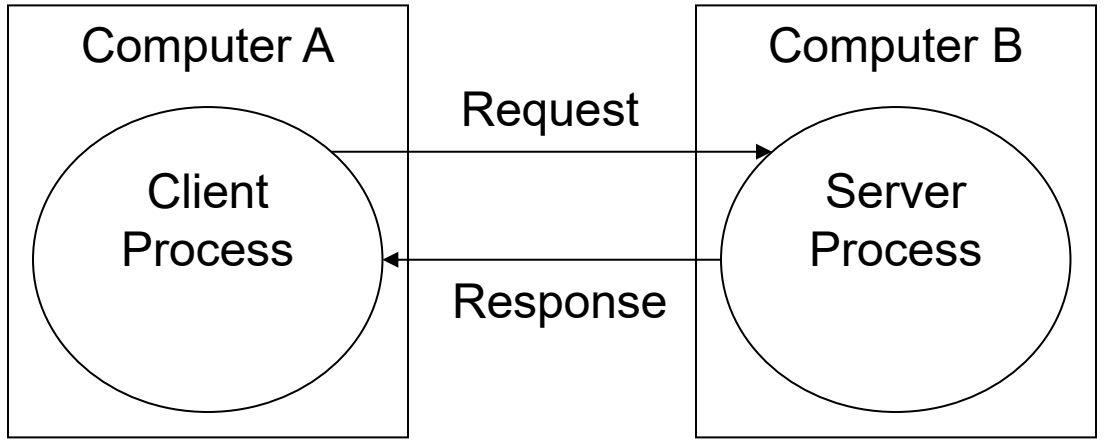
Distributed Architectures

- The principal distributed architectures are
  - Client/Server
    - Most common distributed architecture
    - Servers provide services; Clients make requests of servers to use services
  - Peer-Peer
    - All components play similar role; no distinction between client and server
  - Multi-tier
    - Standard layered design pattern applied to distributed applications

UNIVERSITY OF HULL

Client/Server Architecture

- Client/Server is most common distributed application architecture
- Client and server processes may be on same machine or interact over network



UNIVERSITY OF HULL

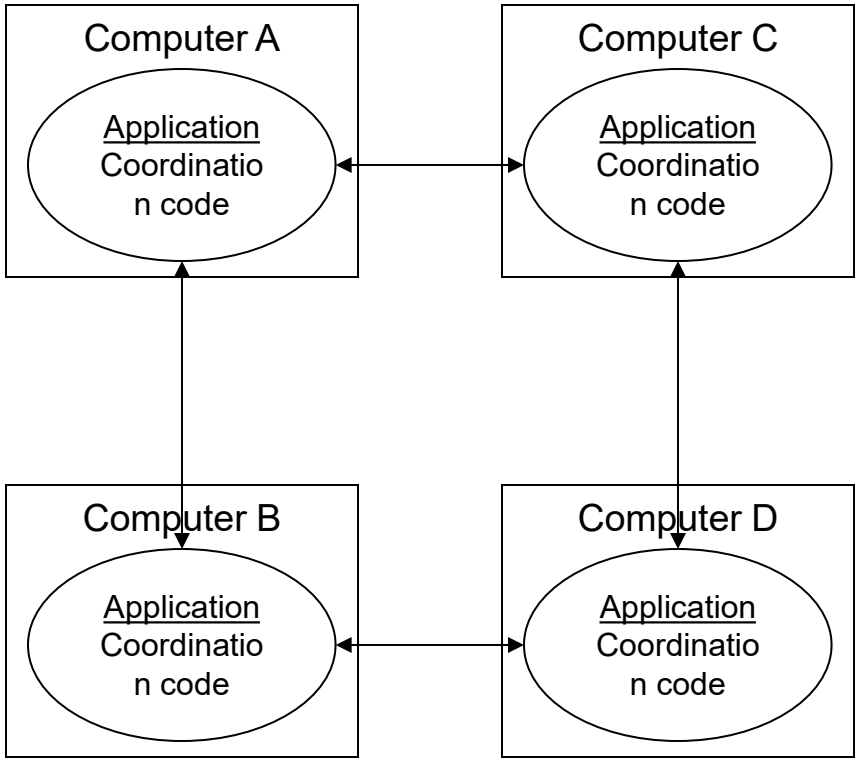
Client/Server - Criticisms

- Can be difficult to achieve performance and scalability goals
  - overheads of resource acquisition and release
  - server becomes performance bottleneck
- May not have key distributed characteristics
  - transparency
  - appearance of single, cohesive application
- Client/Server works well for LAN; not so good for Internet/WAN

UNIVERSITY OF HULL

Peer-Peer Architecture

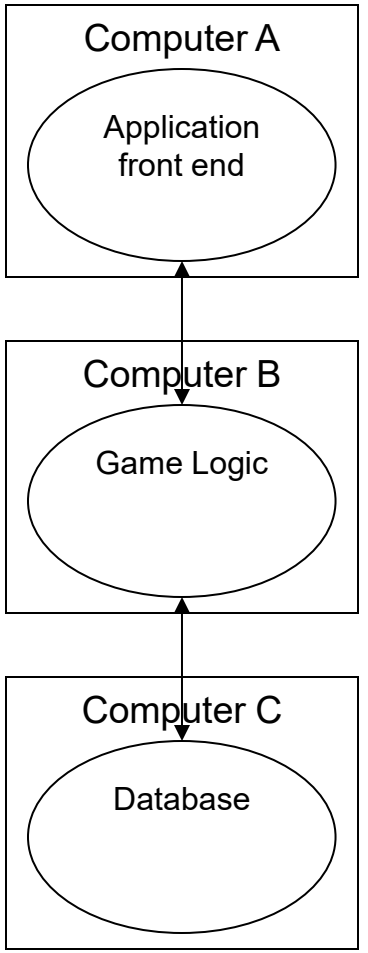
- All components play similar roles
- No distinction between clients and servers
- Peer components interoperate to perform distributed computation
- Each component synchronises application actions and maintains data and application consistency
- Middleware can provide useful services
  - Distributed event mechanisms
  - Group communication facilities



UNIVERSITY OF HULL

Multi-tier Architectures

- Multi-tier distributed architectures are really just application of the sound software design practice of layering to the development of distributed applications
- Different layers may reside on different machines in the network
- No hard and fast rules exist



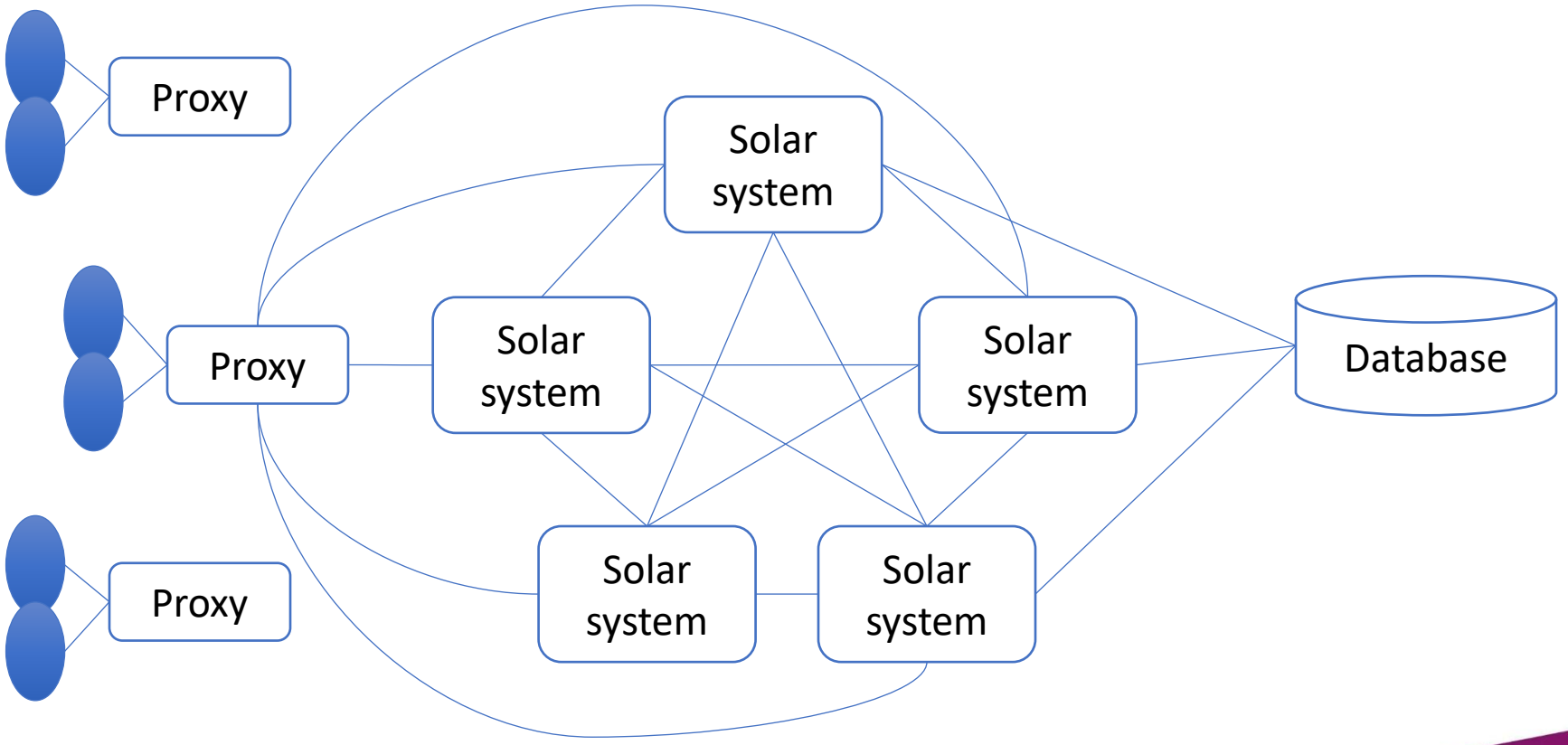
UNIVERSITY OF HULL

Layered Architecture - Benefits

- Benefits of layering include
  - Division of application logic into logically related layers
  - Isolation of layers through well-defined interfaces
  - Ease of modification
  - Reuse of layers across applications
  - Ease of dealing with transactional computations in stateless manner

UNIVERSITY OF HULL

Eve-online server architecture



UNIVERSITY OF HULL

Other Architectural Components

- Caches
  - store of commonly used data located closer to user than data itself
  - exploits Principle of Locality and overcomes network latencies
- Proxies
  - stand-in for a remote object or resource
  - often used with caches and remote invocation mechanisms
- Discovery Services
  - register of resources in the network
  - allows decisions about which resource to use to be dynamically made at runtime
  - can help improve performance and reliability

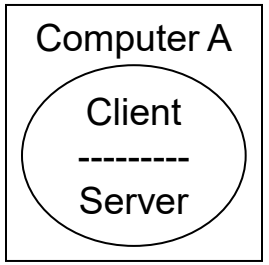
UNIVERSITY OF HULL

Network infrastructure

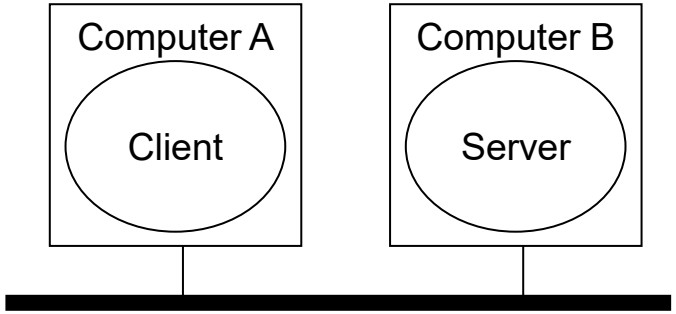
UNIVERSITY OF HULL

Mapping architectures to infrastructure

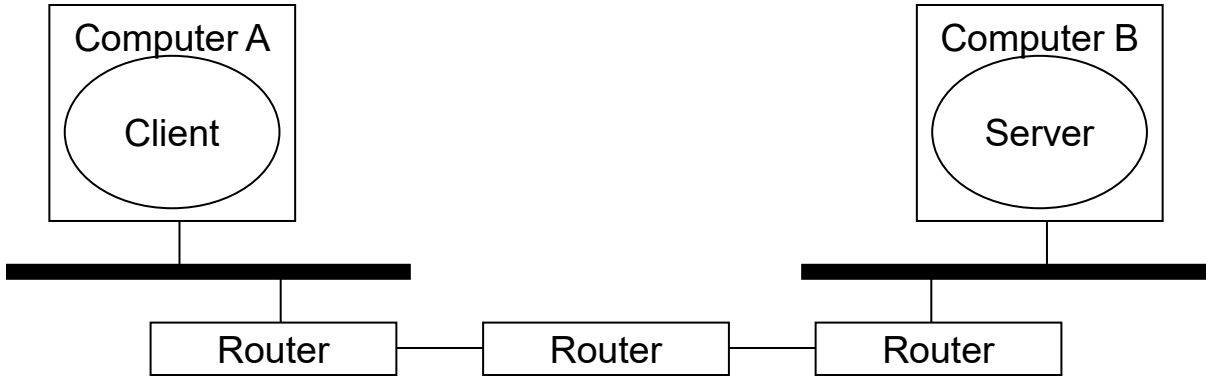
- Local host



- Across LAN



- Across WAN



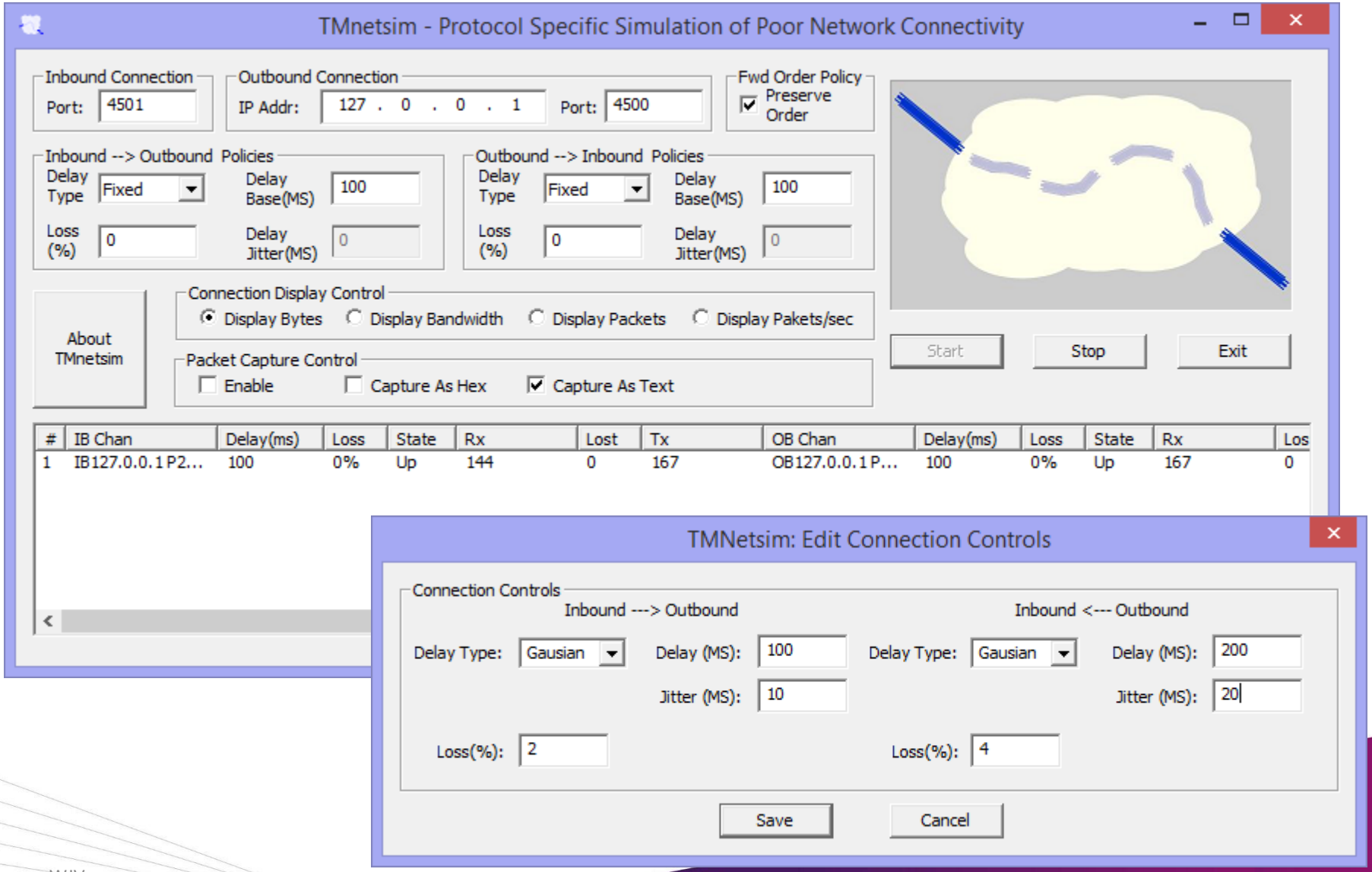
UNIVERSITY OF HULL

Mapping architectures to infrastructure

- Local Host
  - No network overhead
  - No packets dropped and order is perfect
  - Ideal development environment
- Across LAN
  - Network overhead
  - Virtually no packets dropped or out of order
- Across WAN
  - Large performance issues
  - Routers can loose packets, resulting in resends
  - Packets can be duplicated or arrive out of order
  - Considered hostile

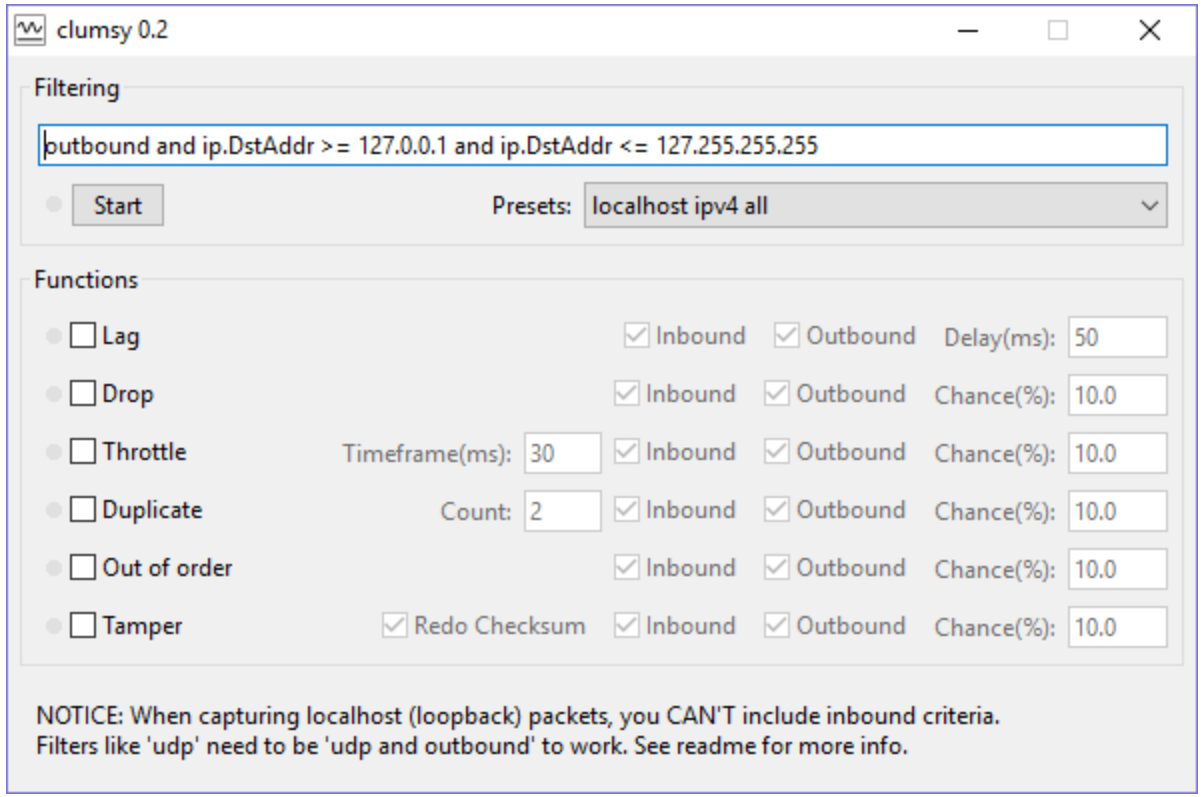
UNIVERSITY OF HULL

TMNetSim



UNIVERSITY OF HULL

Clumsy - MIT



UNIVERSITY OF HULL

Temporal issues

UNIVERSITY OF HULL

Client / server example

- Authoritative server
  - Processes user input
  - Physics
  - AI
  - Game logic
- Dumb client
  - Gets user input
  - Renders scene

UNIVERSITY OF HULL

Typical workflow

- Client
  1. Get time
  2. Get user input
  3. Package data
  4. Send data to server
  5. Receive data from server
  6. Process data
  7. Render scene
  8. Get time
  9. Calculate frame time

- Server
  1. Get time
  2. Receive data from client
  3. Process data
  4. Perform simulation
  5. Package data known to each client
  6. Send data to each client
  7. Get time
  8. Calculate frame time

UNIVERSITY OF HULL

Client side prediction

- Client predicts changes in state
  - Uses server state as the start point
  - Uses same functions as the server
  - Records time and duration of each command
  - Commands are added to a list
- Server processes commands and returns new state
  - Included is a list of the commands that have been run
- Client inspects command list
  - Processed commands are removed from the list
  - Non-processed commands are re-run

UNIVERSITY OF HULL

Client side extrapolation

- Extrapolation places moving objects ahead of their server position, to compensate for latency

UNIVERSITY OF HULL

Client side interpolation

- Interpolation renders moving objects behind their server position, to compensate for latency

UNIVERSITY OF HULL

Server side lag compensation

- The server accurately calculates the amount of lag experienced by each client
- Searches the state history for a time corresponding to when the client sent the command
- Executes command at the correct point in the state history

UNIVERSITY OF HULL

Google Stadia

- Did use ~~Uses~~ “negative latency”

- *Shutdown Jan 2023*

UNIVERSITY OF HULL

Data streams

UNIVERSITY OF HULL

Packaging data

- Data streams (TCP)
  - Low level data transfer mechanisms deposit incoming data in a buffer
  - This are no automatic delimiters to the data, it can be considered a data stream
  - Care has to be taken when reading data from data streams
- Approaches to packaging the data include
  - Fixed length packets
  - Length prefixed packets
  - Delimited packets

UNIVERSITY OF HULL

Object padding

- Consider:

```
1. class Ball {
2.     int _colour;
3.     float _radius;
4.     Vector3f* _position;
5.     ...
6. };
```

- What is the object's size?

UNIVERSITY OF HULL

Object padding

- Never assume you can calculate the amount of padding.
- Two options:
  - Force the compiler to remove all padding
  - Send data members separately
- What about little-endian and big-endian?

UNIVERSITY OF HULL

Data transfer protocols

- JSON
- Protocol Buffers (Protobuf)
- Flat buffers

UNIVERSITY OF HULL

JSON (JavaScript Object Notation)

- Text-based format for storing and exchanging data
- Features
  - Human-readable: Easy for people to read and understand
  - Machine-parsable: Easy for computers to parse and generate
  - Language-independent: Independent of any programming language
  - Open standard: An open data interchange format
- Use cases
  - To transmit data between a server and a web application
  - To store and organize site content
  - Data interchange
  - Configuration management and data transmission

UNIVERSITY OF HULL

Protocol Buffers (Protobuf)

- Free, open-source format for serializing data
  - Define the structure of data in a .proto file
  - Use a program to generate source code from the description
  - Use the generated source code to write and read data to and from data streams
- Features
  - Language-neutral
  - Platform-neutral
  - Binary format: Uses a small binary format payload
  - Backward compatible: Uses numbered fields to ensure backward compatibility
  - Built-in validation: Has built-in validation
  - Extensible: Can be extended
- Use case
  - For developing programs that communicate over networks
  - For storing data
  - For writing and reading structured data to and from data streams
- https://github.com/protocolbuffers/protobuf

UNIVERSITY OF HULL

FlatBuffers

- Free, open-source format for serializing data
  - Define the structure of data in a .fbs file
  - Use a program to generate source code from the description
  - Use the generated source code to write and read data to and from data streams
- Benefits
  - Language-neutral
  - Platform-neutral
  - Binary format: Uses a small binary format payload
  - Backward compatible: Uses numbered fields to ensure backward compatibility
  - Built-in validation: Has built-in validation
  - Extensible: Can be extended
  - **Access to serialized data without parsing/unpacking**
  - **Small Footprint**
- Use case
  - For developing programs that communicate over networks
  - For storing data
  - For writing and reading structured data to and from data streams
- https://flatbuffers.dev/

UNIVERSITY OF HULL

Example .fbs

```
1. namespace HiveCommon;

2. struct Vec3 {
3.       x:float;
4.       y:float;
5.       z:float;
6. }

7. table Node {
8.       id:uint64;
9.       position:Vec3;
10.      error:float;
11.}

12.root_type Node;
```

UNIVERSITY OF HULL

Encoding message

```
1. // encoding message
2. flatbuffers::FlatBufferBuilder fbb;
3. uint64_t id = 100;
4. const HiveCommon::Vec3 pos {10.0f, 20.0f, 30.0f};
5. const auto node = CreateNode( fbb, id, &pos, 0.0f );
6. fbb.Finish( node );
7. …
8. uint8_t  *buf = fbb.GetBufferPointer();
9. int size = fbb.GetSize();
```

UNIVERSITY OF HULL

Decoding message

```
1. // decode message
2. HiveCommon::Node node = GetNode(buf);
3. int64_t id = node->id();
4. const HiveCommon::Vec3 pos = node->pos();
5. float error = node->error();
```

UNIVERSITY OF HULL

Flatbuffer features

- Data types
  - Fundamental
  - Enumerated types
  - Structs
  - Tables
  - Vectors
  - Unions
  - Nested messages
- Optionals
  - Access data from stream without decoding
  - Mutable objects: Update data in stream without decoding
  - Decode and create C++ classes for each Table
  - Append buffer sizes to the data stream
  - Compiler supports: C, C++, C#, Go, Java, Javascript, Lua, PHP, Python, Rust, ….